

9.6 Ollama API

Tools are always connected to the Ollama platform via standardized REST interfaces, regardless of whether Open WebUI or Continue is used. These can be used not only via tools but also directly in your own applications.

Ollama provides a choice of two REST APIs that you can use to connect both your own applications and AI-supported tools. You can either use Ollama's own API (see the following section) or another REST interface that is compatible with the OpenAI API. The latter is currently still in an experimental phase and should enable you to use tools that use the OpenAI API locally in the future.

9.6.1 Ollama REST API

If you run Ollama locally on your system, either directly or in a container, you can access the platform via the address, `http://localhost:11434`. You can communicate with the Ollama API via an HTTP `POST` request to the `/api/generate` path. You can access the OpenAI API via the `/v1` path. In the following code, you can see how to use the Ollama API with Node.js. As this is a simple HTTP communication, you can also implement the example on any other platform.

```
const response = await fetch(
  "http://localhost:11434/api/generate",
  {
    method: "POST",
    body: JSON.stringify({
      model: "llama3.1",
      prompt:
        "Create a TypeScript function that adds two
numbers",
    }),
  }
)
```

```

);

const reader = response.body.getReader();
const decoder = new TextDecoder();

while (true) {
  const { value, done } = await reader.read();

  if (done === true) {
    break;
  }

  const payload = JSON.parse(
    decoder.decode(value, { stream: true })
  );

  process.stdout.write(payload.response);
}

```

The basis for this short example is an HTTP POST request to *<http://localhost:11434/api/generate>*. As a body, you pass an object which allows you to control Ollama and the model used. The two most important properties are `model`, which you use to define the model used (in this case, Llama 3.1), and `prompt`, which contains the request to the model. In addition, you can, for example, deactivate the streaming used by default via `stream: false`.

Streaming, that is, the transmission of individual tokens per package, makes communication with the model somewhat more difficult. However, it also ensures that you receive the answer more quickly, namely as soon as the model has generated the token. To consume the individual tokens and output them to the console, you want to create a reader object with the `getReader` method. You can read a package in an endless loop as soon as it's available with the `reader.read()` method. In this context, the `fetch` API uses binary data, which you have to convert into plain text using the `TextDecoder`. You can write the token generated by the model to the console of your system using `process.stdout.write`.

Save the code in a file named `api.js`. You can then execute the script via `node api.js` and receive the model's response word for word.

In addition to the individual query via the `/api/generate` endpoint, the Ollama API also provides other endpoints:

- `/api/chat`: You can start a chat session with the model via this endpoint.
- `/api/create`: Ollama allows you to create a new model using a model file.
- `/api/blobs/<digest>`: You can use this endpoint to check whether the blob file for creating a new model exists on the server. You can also create a new blob file via this endpoint (POST).
- `/api/tags`: Ollama returns a list of locally available models via this endpoint.
- `/api/show`: You can use this path to display additional details about a model with a POST request.
- `/api/copy`: This endpoint allows you to copy an existing model.
- `/api/delete`: You can use this endpoint to delete an existing model.
- `/api/pull`: You can use this endpoint to load a model from the Ollama library onto your local server.
- `/api/push`: If you're registered with Ollama.ai, you can also upload a model via the Ollama API.
- `/api/embed`: This endpoint enables you to create embeddings from a model.
- `/api/ps`: This endpoint returns a list of the models that are currently loaded into the memory.

As you can see, you can perform all operations via the REST API that you can also perform via the Ollama command line. If you operate Ollama as a server on your local network, you should note that no authentication is required to use the API; that is, the interface is open to everyone. To avoid potential problems, you should install an authentication server that only allows authorized access.

9.6.2 Libraries for Accessing the Ollama API

As direct access to the Ollama APIs can be somewhat cumbersome, there are two official libraries to help you with this: `ollama-js` for JavaScript and `ollama-python` for Python (see [Chapter 10, Section 10.2](#)). These libraries allow you to perform all operations that the REST API also offers, but in the more convenient form of functions.

The following listing shows sample code for `ollama-js` to submit a prompt:

```
import ollama from "ollama";

const response = await ollama.generate({
  model: "llama3.1",
  prompt: "Create a TypeScript function that adds two numbers",
  stream: true,
});

for await (const chunk of response) {
  process.stdout.write(chunk.response);
}
```

`ollama-js` works both on the server side with Node.js and in the browser. To use the library, you must first install the package via the `npm install ollama` command.

By default, `ollama-js` doesn't use streaming, so you can access the return directly via `response.response`. Alternatively, as in this example, you can activate the streaming feature and consume the content via an asynchronous `for` loop.

The advantage of using the libraries provided by Ollama is that your code becomes much more compact, and you have fewer details to worry about.